The functions `assert()` and `require()` are similar in that both check conditions and `revert` the transaction if the check fails. You use `require()` for common validations such as checking the limits of a variable's value (such as `age >= 18`). You expect `require()` to fail sometimes; that is reasonable. `assert()` is meant for handling exceptions. You expect that this condition should not normally fail. To check the head count at a summer camp, for example, you might use `assert(headcount == 44)`. You don't want this check to fail in the middle of the night! On a more serious note, an `assert()` failure costs a lot more in wasted blockchain gas (execution cost) than `require()` reverting, so be selective about what to use when. Use `assert()` sparingly for managing exceptions. Use `require()` for validation of data, computations, and parameter values.

At this time, you can load listing 3.7, Ballotv4.sol, into the Remix IDE. Review the code to see all the incremental improvements (modifiers, `require()`, `revert()`, and `assert()`), and explore its workings.

3.6 Best practices

Now that you've learned about some significant additional features specific to blockchain application development, it's an appropriate time to review some best practices: